

Pearls AQI Predictor

A three-day air quality forecasting system
for Ghotki, Sindh

Shivam Kumar

Institute of Business Administration, Karachi

10Pearls Shine Internship

Data Science Track

Dashboard 10pearls-aqi-predictor-52wav2lhgysbpemqnqu63e.streamlit.app

Repository github.com/Shivam-kumar-30012/10Pearls-AQI-Predictor

Date September 2026

Abstract

This report documents an end-to-end machine learning system that forecasts the Air Quality Index for Ghotki, Sindh, for every hour of the next three days. The system collects data hourly, retrains nightly, and serves predictions through a deployed dashboard, all without manual intervention.

Three models cover the forecast horizon — one per day, each predicting all 24 hours of its block. On a held-out chronological test period they reach R^2 of 0.932, 0.716 and 0.613 at one, two and three days ahead, beating a persistence baseline by a margin that widens with distance.

Particular attention was given to the target variable. The AQI is computed to the EPA standard — correct averaging windows, truncation before band lookup, and extrapolation rather than clamping above scale — and validated independently of model performance, since the target bounds what any model can achieve. Section 4 sets out the method and that validation in full.

What this report covers

- **Sections 1–3** set out the objective, the system architecture, and how two independent data sources were combined without misalignment.
- **Section 4** sets out how the AQI target is computed, why each step of the EPA method is mandatory, and how the implementation was validated.
- **Sections 5–6** describe the 71 engineered features and the exploratory analysis that justified them.
- **Section 7** covers modelling: problem formulation, leakage prevention, the persistence baseline, algorithm comparison, three experiments that did not work, and the final results.
- **Sections 8–10** cover explainability, the automated pipelines and their observed failure modes, and the dashboard.
- **Sections 11–12** state the limitations plainly and draw the conclusion.

A reader wanting only the outcome should read this abstract and Section 7.7. A reader interested in the engineering should begin at Section 4.

Contents

Abstract	1
1 Objective and scope	3
2 System architecture	3
3 Data	3
3.1 Sources	3
3.2 Combining two sources	4
3.3 Gaps and the hourly grid	4
4 Computing the AQI: method and validation	4
4.1 The EPA method	4
4.2 Validating the implementation	5
4.3 Practices adopted as a result	6
5 Feature engineering	7
6 Exploratory analysis	7
7 Modelling	8
7.1 Problem formulation	8
7.2 Preventing leakage	8
7.3 The persistence baseline	8
7.4 Algorithm comparison	9
7.5 Experiments that did not work	9
7.6 Final results	10
8 Explainability	10
9 Automation and reliability	11
9.1 Scheduled pipelines	11
9.2 Observed platform limitations	11
9.3 Degradation behaviour	11
10 The dashboard	12
11 Limitations and further work	12
12 Conclusion	12

1 Objective and scope

The brief specified a serverless machine learning pipeline with five stages: fetch raw data from a public API, engineer features into a feature store, train and evaluate models, register the best of them, and serve predictions through a web application — with the feature pipeline running hourly and training running daily.

Ghotki was selected as the target city. It is a district in northern Sindh with no dedicated monitoring station of its own, which makes a model-based forecast more useful there than in a city with dense instrumentation.

Everything specified was built. Beyond the core requirement, the system also includes SHAP explainability, hazardous-AQI alerting, and a documented set of negative results — experiments that were run properly and did not work, reported here rather than omitted.

2 System architecture

Stage	Implementation
Raw collection	OpenWeather Air Pollution API, Open-Meteo ERA5
Feature engineering	71 engineered features, EPA-compliant AQI
Feature store	Hopsworks, feature group <code>aqi_ghotki_features</code> v3
Training	Ridge, Random Forest, XGBoost — compared per horizon
Registry	Hopsworks Model Registry, currently version 9
Automation	GitHub Actions: hourly collection, nightly training
Serving	Streamlit Community Cloud

The pipeline is a sequence of numbered scripts, each doing one thing and writing its output to disk before the next reads it:

```
01.backfill_pollutants → 02.backfill_weather → 03.merge_raw → 04.build_features
→ 05.upload_to_store → 06.train / 06b.train_delta → 07.register_models
→ 08.hourly_collector → 09.predict → 10.build_snapshot → 11.explain
```

Separating collection from transformation was a deliberate early choice. Downloading five years of hourly data takes several minutes and is rate-limited; feature engineering takes seconds. Keeping them apart meant that every subsequent fix to the feature logic — including the one described in Section 4 — cost thirty seconds rather than an hour.

3 Data

3.1 Sources

Two APIs, joined on an hourly UTC timestamp.

OpenWeather Air Pollution History supplies PM_{2.5}, PM₁₀, NO₂, O₃, CO, SO₂ and NH₃ from 27 November 2020, the earliest date its archive covers for this location.

Open-Meteo ERA5 supplies temperature, humidity, pressure, wind speed, wind direction, cloud cover and precipitation. It was chosen over OpenWeather’s weather endpoint because the free tier of the latter reaches back only about five days, which is useless for building a training set. ERA5 is a reanalysis product — a physics model run backwards over historical observations — and is the standard source for historical weather in research contexts.

3.2 Combining two sources

Mixing sources introduces alignment risk, so three things were enforced. Both feeds are requested in UTC and floored to the hour, removing any possibility of a timezone offset silently misaligning the join. The join is an inner join, so a row exists only where both pollutant and weather data are present — a left join would have left weather columns empty on unmatched rows, which then propagates silently into every derived feature. And OpenWeather’s -9999 sensor-failure sentinel is converted to a missing value at fetch time rather than being allowed to reach a calculation.

The merge yielded **49,017** hours from 50,016 weather readings and 49,017 pollutant readings. ERA5 has no gaps because it is a model; OpenWeather has real outages.

3.3 Gaps and the hourly grid

The merged data was missing 984 hours (1.97%), spread across 31 gaps, the largest lasting 121 hours.

This matters more than the percentage suggests. Lag features are built with a row shift, so `shift(24)` means “24 rows back”, not “24 hours back”. Across a 121-hour gap, a feature named `aqi_lag_24h` would silently contain a value from five days earlier.

The fix is to reindex onto a complete hourly grid before building any feature, so every row is exactly one hour apart. Gaps of three hours or less are filled by time-weighted interpolation; longer ones are left missing and their rows drop out later. The threshold is a judgement: estimating two missing hours between known values is reasonable, inventing five days of weather is not.

A detail worth noting is that interpolation *saves* rows rather than merely patching them. A missing hour does not cost only that hour — it breaks the lag features of every row for the following week. Two interpolated values can preserve around 170 usable rows.

4 Computing the AQI: method and validation

The AQI is the target variable, so its correctness determines the ceiling on everything downstream. This section sets out how it is computed, why each step is mandatory, and the validation that confirmed the implementation was right.

4.1 The EPA method

The index is not a single formula. Each pollutant has its own breakpoint table mapping concentration to an index band; the concentration is looked up, interpolated linearly within its band, and the overall AQI is the *maximum* across pollutants — the dominant-pollutant rule, which reflects that air is only as good as its worst constituent.

Three requirements in the standard are easy to overlook, and each was found to matter materially.

1. Concentrations must be averaged first

EPA breakpoints are defined for *averaged* concentrations, not instantaneous readings:

Pollutant	Averaging window
PM _{2.5} , PM ₁₀	24 hours
CO, O ₃	8 hours

The windows are not arbitrary. They match the exposure period each pollutant’s health standard is built on — particulate harm accrues over a day, ozone and CO act within hours.

The first implementation passed raw hourly readings into these tables. The reasoning at the time was that hourly resolution was the point of an hourly forecast, and averaging appeared to discard information. That reasoning does not hold, for a specific and instructive reason. Hourly concentrations are far more volatile than their 24-hour means, and because the overall AQI takes the maximum across pollutants, that volatility caused the dominant pollutant to flip between PM_{2.5}, PM₁₀ and O₃ from one hour to the next. Consecutive target values were therefore drawn from unrelated measurements, and the series behaved as though the air itself were erratic.

2. Concentrations must be truncated before lookup

EPA publishes bands as (0.0, 12.0), (12.1, 35.4), and so on. Read literally these appear to leave gaps, and a direct implementation of the published tables will find values that match no band at all.

They are not gaps. The standard requires the concentration to be *truncated* — not rounded — to a fixed number of decimals before lookup: one decimal for PM_{2.5} and CO, zero for PM₁₀ and ozone in ppb. After truncation, 12.05 becomes 12.0 and falls cleanly inside the first band. The apparent gaps close entirely.

Truncation rather than rounding matters: `round(12.05, 1)` gives 12.1, which lands in the *next* band and returns a different index.

3. Above-scale readings should extrapolate, not clamp

EPA’s tables end at defined maxima — 604 µg/m³ for PM₁₀, for instance. Ghotki experiences dust events that exceed this; 158 such readings exist in the data.

Clamping them all to 500 is the obvious handling, but it destroys the gradient: a severe dust event and a catastrophic one become the same number, and the model loses any ability to distinguish them. The implementation instead extrapolates along the final band’s slope, which preserves the ordering. Values above 500 are retained for training and clipped only for display.

4.2 Validating the implementation

Because the target bounds every downstream result, the calculator was validated directly rather than inferred from model performance. The two are easily conflated, and a model can only ever be as good as its label allows.

Two checks were run. The first evaluates the calculator at band boundaries, where any departure from the standard surfaces first:

Input	Truncated to	AQI
PM _{2.5} = 12.05	12.0	50
PM _{2.5} = 35.45	35.4	100
O ₃ = 107 µg/m ³	54 ppb	50
PM ₁₀ = 1,262	1,262	1,158 (extrapolated)
Typical Ghotki reading	—	99 (PM _{2.5})

These cases are asserted as tests in the source, so the behaviour is locked in rather than merely observed once.

The second check examines the statistical behaviour of the resulting series against what the physical process implies. Air quality changes gradually, so a correctly computed index should be smooth and strongly autocorrelated:

Property of the computed series	Value
Median hourly change	1
Mean hourly change	1.5
Hourly changes above 100	0
Correlation, today vs. 24 hours later	0.797
Dominant pollutant, PM _{2.5}	89.7% of hours

All five behave as the physics predicts. The correlation figure is particularly useful: squared, it approximates the best R^2 any model could achieve against this target, which sets a realistic expectation of **0.66** at 24 hours before any model is trained. Section 7 reports performance against that expectation.

The dominant-pollutant split is a further sanity check. PM_{2.5} leading 89.7% of hours, PM₁₀ 7.1% during dust events and ozone 3.1% is a physically sensible profile for rural Sindh; a result distributed evenly across pollutants would indicate the averaging windows had not been applied.

4.3 Practices adopted as a result

Two practices were adopted for the remainder of the project.

Regression tests on the calculator. The specific boundary values in the table above are asserted in `src/aqi_calculator.py` and run whenever the file is executed directly. Any future edit that reintroduces the direct-lookup behaviour fails immediately and visibly.

Baseline comparison on every result. Because the target itself had proven capable of setting an invisible ceiling, no subsequent score was accepted without a naive comparison beside it. That is why the persistence baseline appears in every results table in Section 7, rather than only in a summary.

5 Feature engineering

The feature table holds **48,710** rows and **71** features.

Group	Count	Examples
Time and season	13	hour, month, four season indicators
Cyclical	4	<code>hour_sin</code> , <code>month_cos</code>
AQI lags	8	1 h through 168 h
AQI rolling	9	mean, std, max, min over 1 d / 3 d / 7 d
AQI trends	4	6 h, 24 h, 72 h deltas; vs. weekly mean
Pollutants	15	raw, 24 h lag, 24 h rolling mean
Weather	7	temperature, humidity, pressure, wind
Weather derived	11	wind u/v , ventilation, inversion proxy

Three of these deserve explanation.

Wind decomposition. Wind direction in degrees is circular: 359° and 1° are nearly the same direction but maximally distant as numbers. Decomposing into east–west and north–south components fixes this and encodes speed simultaneously. The wind rose in the exploratory analysis showed why it matters — air arriving from the north carries a mean AQI near 160, from the south near 110.

Inversion proxy. `temp_range_24h`, the spread between the day’s highest and lowest temperature, is a proxy for a temperature inversion. A large swing implies clear skies and strong nocturnal cooling, which produces a cold layer near the ground that traps smoke beneath it. It correlates with AQI at $r = +0.30$, and SHAP later confirmed the day +1 model uses it heavily.

Ventilation index. Wind speed weighted by dryness, on the reasoning that wind disperses pollution and dry air does not grow particles. It correlates at only $r = -0.17$ — real but modest, and reported here as such rather than overstated.

Cyclical hour encoding was retained but is likely inert for this target. Because the AQI is computed from 24-hour rolling averages, hour-of-day variation is averaged out by construction, and the exploratory analysis confirms a flat hourly profile. It is noted rather than defended.

6 Exploratory analysis

Eight figures were produced. The findings that shaped the modelling:

Strong seasonality. Winter averages near AQI 200, monsoon near 90, across five clean annual cycles. This justifies the season indicators.

PM_{2.5} dominates. It is the deciding pollutant for 89.7% of hours, PM₁₀ for 7.1% (dust events), ozone for 3.1%. A physically sensible profile for rural Sindh.

Pressure is the strongest weather signal at $r = +0.45$. High pressure implies stagnant air and trapped pollution. Temperature follows at $r = -0.43$.

Bimodal AQI distribution, with peaks near 80 and 155 — consistent with two regimes, clean season and dirty season.

Predictability decays smoothly. Autocorrelation runs 0.997 at one hour, 0.797 at 24, 0.646 at 48, 0.570 at 72. Computed before any model was trained, this set the expectation that results were later compared against, rather than being reverse-justified afterwards.

Time spent in each EPA category: Good 3.3%, Moderate 34.0%, Unhealthy for Sensitive Groups 22.6%, Unhealthy 30.1%, Very Unhealthy 8.3%, Hazardous 1.6%.

7 Modelling

7.1 Problem formulation

The requirement is an hourly forecast across three days — 72 values. Training 72 separate models would be unwieldy to register and maintain, so instead three models each cover a 24-hour block, with `hours_ahead` supplied as a feature.

Each source row is expanded into 24 training examples, one per hour in the block, producing roughly 1.1 million examples per model. The model therefore learns explicitly that a one-hour forecast is near-certain while a 24-hour forecast is not.

7.2 Preventing leakage

The chronological split is applied to the *source row*, not to the expanded examples. Splitting the expanded set would allow 3pm Tuesday to appear in training while 4pm Tuesday appears in test — nearly identical inputs on both sides, producing an inflated score. Splitting by source row keeps all 24 hours of a day together.

Data is never shuffled. The oldest 80% trains, the newest 20% tests, with the boundary at 2 July 2025. Shuffling would let the model learn from 2026 to predict 2021.

7.3 The persistence baseline

Every result is reported against persistence — predicting that the AQI stays exactly as it is now. This matters because AQI is strongly autocorrelated, so persistence is a demanding benchmark rather than a trivial one, and a model can post a high R^2 while contributing nothing.

Horizon	Persistence	Model	Gain
Day +1	0.908	0.932	+0.024
Day +2	0.682	0.716	+0.034
Day +3	0.544	0.613	+0.069

The gain column is the honest measure of what the models contribute, and its trend is the strongest result in the project: the advantage *triples* as the horizon extends. At day +3 persistence has largely collapsed while the models hold, which indicates they have learned genuine atmospheric behaviour rather than simply copying the current value.

7.4 Algorithm comparison

Ridge, Random Forest and XGBoost were compared at every horizon on identical data.

Horizon	Model	R ²	RMSE	MAE
Day +1	XGBoost (delta)	0.932	16.5	8.3
	Ridge	0.930	16.8	10.2
	Persistence	0.908	19.2	9.9
	XGBoost (absolute)	0.889	21.2	9.5
	Random Forest	0.882	21.8	9.8
Day +2	Ridge	0.716	33.9	22.6
	Persistence	0.682	35.8	21.5
	XGBoost	0.652	37.5	21.5
	Random Forest	0.630	38.6	22.3
Day +3	Ridge	0.613	39.5	26.1
	XGBoost	0.595	40.4	26.2
	Persistence	0.544	42.9	26.0
	Random Forest	0.529	43.5	28.4

Ridge winning at the longer horizons is a substantive finding, not a disappointment. Once the AQI target is computed correctly the series is smooth and close to linear in its lag features, leaving little curvature for tree ensembles to exploit. On a noisier target the same outcome would suggest a weak signal; here it indicates that the problem is genuinely linear.

7.5 Experiments that did not work

Three were run properly and are reported because negative results are part of the record.

Delta targets at days 2 and 3. Predicting the change rather than the level helped day +1 substantially — MAE fell from 10.2 to 8.3 — but Ridge scored *identically* at days 2 and 3 (0.7158 vs. 0.7158). This is mathematically expected: for a linear model, predicting $y - x$ when x is already a feature is the same computation as predicting y . XGBoost on the delta target was markedly worse at those horizons (0.606 and 0.484).

Forecast weather instead of observed weather. A methodologically important idea: predicting Saturday’s AQI from Thursday’s weather is wrong, because Saturday’s air depends on Saturday’s conditions. Real forecasting systems train on archived forecasts so that inputs at training time match what is available at prediction time. Open-Meteo’s previous-runs archive was retrieved, but returned only **55.5%** coverage for wind, pressure and humidity at this location — precisely the variables the exploratory analysis identified as most predictive. Adopting it would have cost more than half the training data in exchange for incomplete inputs, so observed weather was retained.

Interaction terms with lead time. Adding `aqi × hours_ahead` and similar products left accuracy essentially unchanged (0.9304 → 0.9296 at day +1) but fixed a real defect: without them the forecast was flat, predicting an identical value for all 24 hours of a block. They were retained for that reason rather than for any gain in score.

7.6 Final results

Horizon	Model	R^2	RMSE	MAE
Day +1	XGBoost (delta)	0.932	16.5	8.3
Day +2	Ridge	0.716	33.9	22.6
Day +3	Ridge	0.613	39.5	26.1

Day-ahead performance is strong: an R^2 of 0.932 and a mean absolute error of 8.3 AQI points, against a series whose own mean is 131. In practical terms the forecast for tomorrow is typically within a tenth of the scale, and comfortably inside the correct EPA category for most hours.

Accuracy declines with lead time, as it must. By day +3 the mean absolute error has risen to 26 points, which is wide enough to straddle a category boundary. The daily average remains considerably more reliable than any individual hour at that distance, which is why the dashboard leads with the daily figure and treats the hourly curve as supporting detail.

The decline is not evidence of a modelling shortfall. Autocorrelation in the series falls from 0.797 at 24 hours to 0.570 at 72 (Section 6), which bounds what is achievable at each horizon; and persistence — the strongest naive benchmark for a slow moving series — itself only reaches RMSE 42.9 at three days. The models beat it by their widest margin precisely there.

8 Explainability

SHAP was applied to all three horizons. It decomposes each prediction into per-feature contributions that sum exactly to the result, so it reports the model’s actual arithmetic rather than an approximation of its reasoning.

Day +1 (XGBoost). Top drivers are $PM_{2.5}$ (3.44), the six-hour trend (1.83), the 24-hour temperature range (1.81), and recent AQI lags. The presence of the temperature range is a useful confirmation — it was added on physical reasoning about inversions, and the model independently found it informative.

Days +2 and +3 (Ridge). Dominated by `aqi_lag_1h` (26.32 at day +2), an order of magnitude above anything else. This is the persistence effect made visible, and it corroborates the modest gain over the persistence baseline at those horizons.

The contrast between the two model types is itself informative: the tree model distributes attention across pollutants and meteorology, while the linear models concentrate almost entirely on the most recent reading.

Two explanation types are produced — global importance averaged over recent forecasts, and a local decomposition of the current prediction — and both are surfaced in the dashboard’s Accuracy view.

9 Automation and reliability

9.1 Scheduled pipelines

Two GitHub Actions workflows. The hourly one collects new readings, builds features and writes to both the local CSV and the feature store. The nightly one collects, retrains all models, registers them, rebuilds the dashboard snapshot, and commits it back to the repository.

The collector fetches a *range* rather than a single hour: it asks each destination for its newest timestamp and retrieves everything since. This is one code path that handles the normal case (one hour) and recovery after an outage (several hundred) identically, and it means a missed run needs no special handling.

9.2 Observed platform limitations

Three were encountered and worked around; all are documented because they shaped the design.

GitHub scheduling is best-effort. On free accounts, hourly crons fire perhaps three or four times a day rather than 24. This is widely reported and is not specific to this project. The range-fetch design absorbs it — data completeness is unaffected, only latency.

Hopworks free-tier reads are unreliable. Row reads intermittently time out after roughly 170 seconds. Every read is therefore retried with growing pauses, and every consumer can fall back to a local CSV.

Large inserts fail materialisation. Uploading several hundred rows at once caused the Spark job that makes rows queryable to fail, leaving them accepted but invisible. Chunking to 100 rows with pauses resolved it, and the chunk size is now configured accordingly.

9.3 Degradation behaviour

The system was designed so that failures degrade rather than break. The behaviour differs between a developer machine and a scheduled run, because the two have different local state, and both cases are set out below.

Failure	On a developer machine	On a scheduled runner
Store read fails, collector	Uses the local CSV for context	Exits; next run retries
Store read fails, training	Falls back to the local CSV	Refuses to train (see below)
Upload fails	Data already written locally	Data already written locally
Materialisation deferred	Rows accepted; visible later	Rows accepted; visible later
Scheduled run skipped	—	Next run collects the backlog
Dashboard cannot reach store	Serves the committed snapshot	Serves the committed snapshot

The second row deserves comment. A developer machine holds the full 48,710-row feature file, so falling back to it is a genuine recovery. A scheduled runner starts with no local data, and although the collector writes a file during the run, it holds only the hours just fetched — falling back to that would mean training on a few dozen rows.

Training therefore refuses to run on fewer than 5,000 rows. The guard was added after a scheduled run trained on 42: the runner’s freshly written file carried a newer timestamp than the store and won a comparison that considered recency alone. Selection now weighs recency *and* completeness, with the row-count floor as an independent second check. When it triggers, the run fails visibly and the registered models stay in service — the correct outcome, since a stale model beats one trained on an unrepresentative sample.

The dashboard is unaffected by any of this: it reads a committed snapshot rather than querying the store, so a failure upstream leaves it serving the last good forecast, with the issue time shown in the footer.

10 The dashboard

Deployed on Streamlit Community Cloud with four views: current conditions and a 72-hour overview; a forecast view offering either a full day or any single hour; model accuracy with SHAP explanations; and 30 days of history.

The dashboard reads a committed JSON snapshot rather than querying the feature store live. This was a deliberate reversal of an earlier design. A live-fetch path existed at first, but it could not produce anything newer — the snapshot is rebuilt from the same models in the same nightly run, so a live read returns the same forecast several seconds slower, and during a Hopsworks outage it froze the page for minutes. Reading a committed file cannot fail.

Hazardous-AQI alerting is implemented as a banner that appears whenever any forecast hour reaches AQI 150 or above, naming the peak value, its category, and when it occurs.

11 Limitations and further work

Day +3 accuracy. At three days the mean absolute error reaches 26 AQI points, wide enough to cross a category boundary. Seven distinct approaches were tried and all converged on a similar figure, which suggests the constraint is the information available rather than the method applied.

Observed rather than forecast weather. The models use the current day’s weather to predict future AQI, where forecast weather for the target day would be correct. The archive available at this location was too sparse to use. A paid forecast archive would likely improve days +2 and +3 most.

No upwind data. Ghotki’s air is partly transported from elsewhere. Pollutant readings from Sukkur, Larkana or Rahim Yar Khan, combined with wind direction, would add genuinely new information rather than rearranging what is already present. This is the most promising remaining avenue.

Sparse coverage of extreme events. Only 1.6% of hours fall in the Hazardous category, so the models see few examples of exactly the conditions where a forecast matters most.

Dropped-row bias. Around 4% of rows are excluded for incomplete features. Their mean AQI is 110 against 132 for retained rows — the exclusions skew towards cleaner periods, so reported accuracy is if anything slightly conservative.

12 Conclusion

A complete forecasting system was built and deployed, meeting the specified architecture at every stage and running unattended: hourly collection, nightly retraining, versioned model registration, and a served dashboard.

Two decisions shaped the outcome more than any choice of algorithm. The first was treating the target variable as something to be validated in its own right. The AQI is computed rather than measured, and a computed target places a ceiling on every model trained against it — one no amount of model comparison will reveal, because every model sits below it equally. Verifying the implementation against the EPA standard established that ceiling before any modelling began.

The second was reporting every result against a persistence baseline. Air quality is strongly autocorrelated, so a model can post a high R^2_{12} while contributing little of its own. The gap between model and baseline is the honest measure, and it widens from +0.024 at one day to +0.069 at three — the models earn more of their keep precisely where forecasting is hardest. Both practices cost far less than the modelling work they informed.